SMART QUIZ WEB APPLICATION

A Complete Computer Science Project Report & Source Code Listing

Curriculum: Class 11 Computer Science | **Academic Year:** 2026-2027 | **Tech Stack:** Python, Flask, MySQL, SQLite, HTML5, CSS3, JavaScript

---

1. ABSTRACT

The **Smart Quiz Web Application** is an interactive, multi-category educational platform engineered to assess students' conceptual and practical mastery across high school computer science disciplines and general science. Built strictly following Class 11 CBSE and state curriculum specifications, the project integrates a robust Python Flask backend with normalized relational database management systems (supporting both **MySQL** and **SQLite**), delivered through an intuitive, student-friendly web interface.

A foundational design principle of the application is **tamper-proof evaluation integrity**. Traditional client-only quiz scripts expose correct answers and explanations directly within client HTML/JS, allowing students to inspect solutions using browser developer tools. In Smart Quiz, all scoring and answer validations are executed strictly **server-side**. Correct options, score percentages, and educational explanations are dispatched to the client only after formal submission.

Core Architectural Features:

- **Dual Database Engine:** Auto-connects to standard MySQL databases with a zero-configuration fallback to an embedded SQLite database (`smart_quiz.db`) for dependable evaluation in any environment.
- **Randomized 10-Question Tests:** Each test randomly selects 10 questions from a bank of 132+ questions across six subjects: *Java, Python, MySQL, DBMS, History, and Science*.
- **Balanced Option Distribution:** Shuffled question options guarantee an equitable ~25% distribution across keys A, B, C, and D, preventing predictable answering patterns.
- **Relational Leaderboard & History:** Tracks student test history, scores, and completion timestamps in database tables with live real-time search filtering.
- **Cross-Platform Responsiveness:** Fully responsive design featuring a mobile hamburger drawer, touch targets (≥ 48px), and public accessibility over local Wi-Fi and cloud.

2. HARDWARE AND SOFTWARE REQUIREMENTS

The application is engineered with high lightweight efficiency, enabling execution across standard school laboratory computers, cloud servers, and mobile touchscreen devices.

2.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification	Purpose & Role in Project
Processor (CPU)	Intel Core i3 (2nd Gen) / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 or higher (2.4+ GHz)	Executes Python Flask server, SQL query resolution, and concurrent client requests.
Memory (RAM)	2 GB DDR3	4 GB - 8 GB DDR4	Accommodates OS, Python runtime, database caching, and in-memory session pools.
Storage (Disk)	200 MB free space	500 MB SSD free space	Stores application code, relational database (`smart_quiz.db`), CSS, JS, and server logs.
Network Adapter	Standard Ethernet or Wi-Fi (802.11n)	Gigabit Ethernet / Wi-Fi 5 (802.11ac)	Enables local network binding (0.0.0.0) for multi-device access across lab computers & phones.
Display / Client	1024 × 768 resolution	1920 × 1080 Full HD (PC) or 412 × 892 (Android)	Presents responsive user interface across desktop monitors, laptops, and mobile screens.

2.2 Software Requirements

Layer / Category	Software / Tool	Version / Specification	Functionality in Project
Operating System	Microsoft Windows / Linux / macOS	Windows 10 / 11, Ubuntu 20.04+, macOS 12+	Host environment for server execution and database management.
Programming Language	Python	Python 3.10 or 3.11+	Core backend language implementing server routing and scoring algorithms.
Web Framework	Flask	Flask 3.1+ (Jinja2 templating engine)	Handles HTTP routes, cookie-signed sessions, and template rendering.
Database Management	MySQL Server (with SQLite Fallback)	MySQL 8.0+ / SQLite 3	Relational storage for normalized entities: categories, questions, users, results.
Database Drivers	PyMySQL & Python sqlite3	PyMySQL 1.1+ (Python DB-API 2.0)	Facilitates safe, parameterized SQL execution preventing SQL injection.
Frontend UI & Styling	HTML5, Vanilla CSS3, JavaScript (ES6+)	Standards-compliant modern web stack	Constructs responsive layouts, modals, validation, and micro-animations.
Production WSGI	Waitress & Gunicorn	Waitress 3.0+ / Gunicorn 21.2+	Production-grade WSGI servers supporting concurrent student attempts.
Client Web Browser	Modern Web Browser	Google Chrome, Microsoft Edge, Safari, Firefox	Executes frontend UI on desktop computers, Chromebooks, and Android mobile devices.

3. SYSTEM ARCHITECTURE & DBMS DESIGN

The database architecture strictly adheres to **Third Normal Form (3NF)** principles studied in Class 11. The relational schema divides system state into four logical relations:

1. **categories:** Stores subject categories (Java, Python, MySQL, DBMS, History, Science) with unique codes and UI icons.
2. **questions:** Stores multiple-choice question records, options A–D, correct keys, and explanations, linked to categories via a Foreign Key with ON DELETE CASCADE.
3. **users:** Stores student session identifiers and names without requiring sensitive personal data or passwords.
4. **quiz_results:** Records completed quiz outcomes, scores, percentages, and timestamps, referencing both users and categories.

To guarantee high query throughput during concurrent tests, indexed lookup structures are defined on category_id, user_id, and completion_date.

4. DATABASE DDL SCHEMA CODE

The following SQL script (schema.sql) creates the relational database, table structures, referential constraints, and performance indexes for MySQL and SQLite:

File: schema.sql SQL Data Definition Language (DDL)

```
-- ======================================================
-- Smart Quiz Database Schema
-- Database Name: smart_quiz
-- Suitable for 11th-Standard School Project
-- ======================================================

-- Create the database if it doesn't already exist
CREATE DATABASE IF NOT EXISTS smart_quiz CHARACTER SET utf8mb4 COLLATE
  utf8mb4_unicode_ci;
USE smart_quiz;

-- ------------------------------------------------------
-- 1. Table: categories
-- Stores quiz subject categories (Java, Python, MySQL, DBMS, History, Science)
-- ------------------------------------------------------
DROP TABLE IF EXISTS quiz_results;
DROP TABLE IF EXISTS questions;
DROP TABLE IF EXISTS users;
DROP TABLE IF EXISTS categories;

CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    code VARCHAR(50) NOT NULL UNIQUE,
    description TEXT NOT NULL,
    icon VARCHAR(50) NOT NULL DEFAULT 'book',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------
-- 2. Table: questions
-- Stores multiple-choice questions belonging to a category
-- ------------------------------------------------------
CREATE TABLE questions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    question_text TEXT NOT NULL,
    option_a VARCHAR(255) NOT NULL,
    option_b VARCHAR(255) NOT NULL,
    option_c VARCHAR(255) NOT NULL,
    option_d VARCHAR(255) NOT NULL,
    correct_option CHAR(1) NOT NULL, -- 'A', 'B', 'C', or 'D'
    explanation TEXT,
    difficulty VARCHAR(20) DEFAULT 'Easy',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_questions_category FOREIGN KEY (category_id)
        REFERENCES categories(id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
48 |
49 | -- ----------------------------------------------------
50 | -- 3. Table: users
51 | -- Stores simple student registrations (name only)
52 | -- ----------------------------------------------------
53 | CREATE TABLE users (
54 |     id INT AUTO_INCREMENT PRIMARY KEY,
55 |     student_name VARCHAR(100) NOT NULL,
56 |     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
57 | ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
58 |
59 | -- ----------------------------------------------------
60 | -- 4. Table: quiz_results
61 | -- Stores quiz score, percentage, user and category references
62 | -- ----------------------------------------------------
63 | CREATE TABLE quiz_results (
64 |     id INT AUTO_INCREMENT PRIMARY KEY,
65 |     user_id INT NOT NULL,
66 |     category_id INT NOT NULL,
67 |     score INT NOT NULL,
68 |     total_questions INT NOT NULL,
69 |     percentage DECIMAL(5,2) NOT NULL,
70 |     completion_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
71 |     CONSTRAINT fk_results_user FOREIGN KEY (user_id)
72 |         REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE,
73 |     CONSTRAINT fk_results_category FOREIGN KEY (category_id)
74 |         REFERENCES categories(id) ON DELETE CASCADE ON UPDATE CASCADE
75 | ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
76 |
77 | -- Indexes for performance optimization
78 | CREATE INDEX idx_questions_category ON questions(category_id);
79 | CREATE INDEX idx_results_user ON quiz_results(user_id);
80 | CREATE INDEX idx_results_category ON quiz_results(category_id);
81 | CREATE INDEX idx_results_date ON quiz_results(completion_date);
```

5. BACKEND SOURCE CODE (PYTHON & FLASK)

The backend layer comprises the main Flask application controller (app.py), the database abstraction manager (database.py), the automated test suite (test_app.py), and the production WSGI runner (run_production.py).

5.1 Flask Application Controller (app.py)

Implements route management, session state tracking, JSON APIs, and secure server-side score calculation:

File: app.py — Flask Application & Routes

```
"""
Smart Quiz - Flask Web Application
Backend application for 11th-Standard School Project.
Provides routing, session management, secure server-side scoring,
and integrates with MySQL / database module.
"""

import os
from flask import (
    Flask,
    render_template,
    request,
    redirect,
    url_for,
    session,
    flash,
    jsonify
)
from database import db

app = Flask(__name__)
# Secret key for secure session cookies
app.secret_key = os.getenv("SECRET_KEY", "smart-quiz-school-project-secret-key-2026")


@app.context_processor
def inject_global_data():
    """Injects database status and common variables to all templates."""
    return {
        "db_engine": db.engine,
        "current_student": session.get("student_name", "")
    }


@app.route("/")
def index():
    """Home Page: displays title, subtitle, welcome message, stats, and 6 category cards."""
    try:
        categories = db.get_all_categories_with_count()
        stats = db.get_system_stats()
    except Exception as e:
        flash(f"Error loading categories: {str(e)}", "danger")
        categories = []
        stats = {}
    return render_template("index.html", categories=categories, stats=stats)

```

```
47 |
48 | @app.route("/start-quiz/<int:category_id>", methods=["POST"])
49 | def start_quiz(category_id):
50 |     """
51 |     Validates student name from modal/form, saves in session,
52 |     and initiates the quiz session.
53 |     """
54 |     student_name = request.form.get("student_name", "").strip()
55 |     if not student_name:
56 |         flash("Please enter your name to start the quiz.", "warning")
57 |         return redirect(url_for("index"))
58 |
59 |     # Validate category exists
60 |     category = db.get_category_by_id(category_id)
61 |     if not category:
62 |         flash("Invalid category selected.", "danger")
63 |         return redirect(url_for("index"))
64 |
65 |     # Store student details in session
66 |     session["student_name"] = student_name
67 |     session["category_id"] = category_id
68 |     session["current_quiz_review"] = None  # Clear past review data
69 |
70 |     return redirect(url_for("quiz", category_id=category_id))
71 |
72 |
73 | @app.route("/quiz/<int:category_id>")
74 | def quiz(category_id):
75 |     """
76 |     Quiz Interface:
77 |     Renders 10 randomly selected questions for the selected category.
78 |     CRITICAL SECURITY RULE: Correct answers are NOT exposed to the client!
79 |     """
80 |     student_name = session.get("student_name") or request.args.get("demo_student")
81 |     if student_name and not session.get("student_name"):
82 |         session["student_name"] = student_name
83 |     if not student_name:
84 |         flash("Please enter your name on the home page before taking a quiz.",
   |           "info")
85 |         return redirect(url_for("index"))
86 |
87 |     category = db.get_category_by_id(category_id)
88 |     if not category:
89 |         flash("Category not found.", "danger")
90 |         return redirect(url_for("index"))
91 |
92 |     # Fetch 10 random questions (without correct_option and without explanation)
93 |     questions = db.get_random_questions(category_id, limit=10)
```

```
    if not questions:
        flash(f"No questions currently available in '{category['name']}'.",
          "warning")
        return redirect(url_for("index"))

    # Store question IDs in session to prevent client manipulation upon submission
    session["active_question_ids"] = [q["id"] for q in questions]
    session["active_category_id"] = category_id

    return render_template(
        "quiz.html",
        category=category,
        questions=questions,
        total_questions=len(questions),
        student_name=student_name
    )


@app.route("/submit-quiz", methods=["POST"])
def submit_quiz():
    """
    Submits student answers, calculates score on server side,
    stores result in MySQL/SQLite database, and prepares review data.
    """
    student_name = session.get("student_name")
    if not student_name:
        if request.is_json:
            return jsonify({"success": False, "error": "Session expired. Please
              start again from Home."}), 400
        flash("Session expired. Please enter your name again.", "warning")
        return redirect(url_for("index"))

    category_id = session.get("active_category_id")
    question_ids = session.get("active_question_ids", [])

    if not category_id or not question_ids:
        if request.is_json:
            return jsonify({"success": False, "error": "No active quiz session
              found."}), 400
        flash("No active quiz found. Please choose a category to begin.",
          "warning")
        return redirect(url_for("index"))

    # Parse submitted answers (supports both JSON fetch and traditional Form
      submission)
    if request.is_json:
        payload = request.get_json() or {}
        submitted_answers = payload.get("answers", {})
```

```
    else:
        submitted_answers = {}
        for q_id in question_ids:
            key = f"q_{q_id}"
            if key in request.form:
                submitted_answers[str(q_id)] = request.form[key]

    # Fetch authentic question records (including correct_option and explanation)
    full_questions = db.get_questions_for_scoring(question_ids)
    q_dict = {str(q["id"]): q for q in full_questions}

    score = 0
    total_questions = len(question_ids)
    review_list = []

    for q_id in question_ids:
        str_id = str(q_id)
        q_data = q_dict.get(str_id)
        if not q_data:
            continue

        selected_opt = submitted_answers.get(str_id, "").strip().upper()
        correct_opt = q_data["correct_option"].strip().upper()
        is_correct = (selected_opt == correct_opt)

        if is_correct:
            score += 1

        review_list.append({
            "id": q_data["id"],
            "question_text": q_data["question_text"],
            "option_a": q_data["option_a"],
            "option_b": q_data["option_b"],
            "option_c": q_data["option_c"],
            "option_d": q_data["option_d"],
            "selected_option": selected_opt or "Unanswered",
            "correct_option": correct_opt,
            "is_correct": is_correct,
            "explanation": q_data["explanation"]
        })

    percentage = round((score / total_questions) * 100, 2) if total_questions > 0
      else 0.0

    # Save to database (users and quiz_results tables)
    try:
        user_id = db.get_or_create_user(student_name)
        result_id = db.save_quiz_result(
```

```
            user_id=user_id,
            category_id=category_id,
            score=score,
            total_questions=total_questions,
            percentage=percentage
        )
    except Exception as e:
        print(f"[ERROR] Failed to save quiz result: {e}")
        result_id = 0

    # Save detailed review in session for the results page
    session["current_quiz_review"] = review_list

    if request.is_json:
        return jsonify({
            "success": True,
            "result_id": result_id,
            "redirect_url": url_for("result", result_id=result_id)
        })

    return redirect(url_for("result", result_id=result_id))


@app.route("/result/<int:result_id>")
def result(result_id):
    """
    Results Page:
    Displays score, percentage, correct/incorrect counters,
    motivational message, and full answer review with explanations.
    """
    result_data = None
    if result_id > 0:
        result_data = db.get_result_by_id(result_id)

    # Fallback to session values if direct record lookup fails
    student_name = session.get("student_name", "Student")
    review_list = session.get("current_quiz_review", [])

    if result_data:
        score = result_data["score"]
        total = result_data["total_questions"]
        percentage = float(result_data["percentage"])
        category_name = result_data["category_name"]
        category_code = result_data.get("category_code", "general")
        student_name = result_data["student_name"]
    else:
        # Compute from review_list if available
        total = len(review_list) or 10
```

```
232 |         score = sum(1 for r in review_list if r["is_correct"])
233 |         percentage = round((score / total) * 100, 2) if total > 0 else 0.0
234 |         cat_id = session.get("active_category_id", 1)
235 |         cat_obj = db.get_category_by_id(cat_id)
236 |         category_name = cat_obj["name"] if cat_obj else "Quiz"
237 |         category_code = cat_obj["code"] if cat_obj else "general"
238 |
239 |     incorrect = total - score
240 |
241 |     # Performance-based motivational message
242 |     if percentage >= 90:
243 |         badge_title = "Outstanding Performance! ■"
244 |         badge_message = "Brilliant job! You demonstrated complete mastery of this
    |           subject."
245 |         badge_class = "result-excellent"
246 |     elif percentage >= 70:
247 |         badge_title = "Great Effort! ■"
248 |         badge_message = "Well done! You have a solid grasp of key concepts."
249 |         badge_class = "result-good"
250 |     elif percentage >= 50:
251 |         badge_title = "Good Attempt! ■"
252 |         badge_message = "Nice try! Review the explanations below and give it
    |           another shot."
253 |         badge_class = "result-average"
254 |     else:
255 |         badge_title = "Keep Practicing! ■"
256 |         badge_message = "Don't be discouraged! Check the detailed explanations
    |           below to learn and retake the quiz."
257 |         badge_class = "result-needs-improvement"
258 |
259 |     category_id = session.get("active_category_id", 1)
260 |
261 |     return render_template(
262 |         "result.html",
263 |         result_id=result_id,
264 |         student_name=student_name,
265 |         category_name=category_name,
266 |         category_id=category_id,
267 |         category_code=category_code,
268 |         score=score,
269 |         total=total,
270 |         incorrect=incorrect,
271 |         percentage=percentage,
272 |         badge_title=badge_title,
273 |         badge_message=badge_message,
274 |         badge_class=badge_class,
275 |         review_list=review_list
276 |     )
```

```
277 |
278 |
279 | @app.route("/history")
280 | def history():
281 |     """Displays recent quiz history, leaderboard, and overall statistics."""
282 |     recent_results = db.get_recent_results(limit=30)
283 |     stats = db.get_system_stats()
284 |     return render_template("history.html", results=recent_results, stats=stats)
285 |
286 |
287 | @app.route("/about")
288 | def about():
289 |     """
290 |     Project documentation page explaining DBMS concepts, SQL schema,
291 |     Python Flask structure, and Java topics for school project evaluation.
292 |     """
293 |     return render_template("about.html")
294 |
295 |
296 | @app.route("/api/categories")
297 | def api_categories():
298 |     """JSON API to fetch all categories."""
299 |     categories = db.get_all_categories_with_count()
300 |     return jsonify(categories)
301 |
302 |
303 | @app.errorhandler(404)
304 | def page_not_found(e):
305 |     return render_template("404.html"), 404
306 |
307 |
308 | @app.errorhandler(500)
309 | def internal_server_error(e):
310 |     return render_template("500.html", error=str(e)), 500
311 |
312 |
313 | import socket
314 |
315 |
316 | def get_local_ip():
317 |     try:
318 |         s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
319 |         s.connect(("8.8.8.8", 80))
320 |         ip = s.getsockname()[0]
321 |         s.close()
322 |         return ip
323 |     except Exception:
324 |         return "127.0.0.1"
```

```
325 |
326 |
327 | if __name__ == "__main__":
328 |     local_ip = get_local_ip()
329 |     print("=" * 60)
330 |     print(" Smart Quiz Web Application")
331 |     print(f" Database Engine: {db.engine.upper()}")
332 |     print(f" Local (PC/Desktop):      http://127.0.0.1:5000")
333 |     print(f" Android / Mobile Device: http://{local_ip}:5000")
334 |     print("=" * 60)
335 |     port = int(os.environ.get("PORT", 5000))
336 |     # Bind to 0.0.0.0 so phones and tablets on the same network or cloud routers
    |       can access it
337 |     app.run(host="0.0.0.0", port=port, debug=False, use_reloader=False)
```

5.2 Database Operations & Fallback Manager (database.py)

Manages MySQL connections with automatic fallback to SQLite, parameterized queries, and seeding:

File: database.py Database Operations Module

```
"""
Smart Quiz - Database Operations Module
Handles MySQL connections, schema execution, parameterized queries,
and includes a transparent SQLite fallback for immediate testing/demonstration
if MySQL server is not locally running.
"""

import os
import sqlite3
import pymysql
from pymysql.cursors import DictCursor
from seed_data import seed_database

# Configuration - can be overridden via environment variables
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_PORT = int(os.getenv("DB_PORT", 3306))
DB_USER = os.getenv("DB_USER", "root")
DB_PASSWORD = os.getenv("DB_PASSWORD", "")
DB_NAME = os.getenv("DB_NAME", "smart_quiz")

SQLITE_DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)),
  "smart_quiz.db")


class DatabaseManager:
    def __init__(self):
        self.engine = None  # 'mysql' or 'sqlite'
        self.mysql_error = None
        self._test_connection()
        self.init_database()

    def _test_connection(self):
        """Attempts to connect to MySQL; falls back to SQLite if unreachable."""
        try:
            conn = pymysql.connect(
                host=DB_HOST,
                port=DB_PORT,
                user=DB_USER,
                password=DB_PASSWORD,
                cursorclass=DictCursor,
                connect_timeout=3
            )
            conn.close()
            self.engine = "mysql"
            print(f"[DATABASE] Connected successfully to MySQL at
              {DB_HOST}:{DB_PORT}.")
        except Exception as e:
            self.engine = "sqlite"
```

```
            self.mysql_error = str(e)
            print(f"[DATABASE] MySQL not reachable ({e}). Switching seamlessly to
              SQLite mode ({SQLITE_DB_PATH}).")

    def get_connection(self):
        """Returns an active database connection based on the chosen engine."""
        if self.engine == "mysql":
            try:
                return pymysql.connect(
                    host=DB_HOST,
                    port=DB_PORT,
                    user=DB_USER,
                    password=DB_PASSWORD,
                    database=DB_NAME,
                    cursorclass=DictCursor,
                    autocommit=True
                )
            except pymysql.err.OperationalError as e:
                # Database might not exist yet; connect to root to create it
                if e.args[0] == 1049:  # Unknown database
                    return pymysql.connect(
                        host=DB_HOST,
                        port=DB_PORT,
                        user=DB_USER,
                        password=DB_PASSWORD,
                        cursorclass=DictCursor,
                        autocommit=True
                    )
                raise
        else:
            conn = sqlite3.connect(SQLITE_DB_PATH)
            conn.row_factory = sqlite3.Row  # Enables dict-like row access
            conn.execute("PRAGMA foreign_keys = ON;")
            return conn

    def init_database(self):
        """Initializes tables and seeds initial data if empty."""
        if self.engine == "mysql":
            self._init_mysql()
        else:
            self._init_sqlite()

        # Seed categories and questions if database is empty
        cat_count = self.execute_query("SELECT COUNT(*) AS count FROM categories",
          fetch_one=True)
        count_val = cat_count.get("count", 0) if isinstance(cat_count, dict) else
          (cat_count[0] if cat_count else 0)
        if count_val == 0:
```

```
            print("[DATABASE] No categories found. Automatically seeding questions...")
            seed_database(self)
        else:
            print(f"[DATABASE] Database ready with {count_val} categories.")

    def _init_mysql(self):
        """Creates smart_quiz database and MySQL schema."""
        conn = pymysql.connect(
            host=DB_HOST,
            port=DB_PORT,
            user=DB_USER,
            password=DB_PASSWORD,
            cursorclass=DictCursor,
            autocommit=True
        )
        try:
            with conn.cursor() as cur:
                cur.execute(f"CREATE DATABASE IF NOT EXISTS `{DB_NAME}` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;")
                cur.execute(f"USE `{DB_NAME}`;")

                cur.execute("""
                CREATE TABLE IF NOT EXISTS categories (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    name VARCHAR(100) NOT NULL UNIQUE,
                    code VARCHAR(50) NOT NULL UNIQUE,
                    description TEXT NOT NULL,
                    icon VARCHAR(50) NOT NULL DEFAULT 'book',
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
                """)

                cur.execute("""
                CREATE TABLE IF NOT EXISTS questions (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    category_id INT NOT NULL,
                    question_text TEXT NOT NULL,
                    option_a VARCHAR(255) NOT NULL,
                    option_b VARCHAR(255) NOT NULL,
                    option_c VARCHAR(255) NOT NULL,
                    option_d VARCHAR(255) NOT NULL,
                    correct_option CHAR(1) NOT NULL,
                    explanation TEXT,
                    difficulty VARCHAR(20) DEFAULT 'Easy',
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    CONSTRAINT fk_questions_category FOREIGN KEY (category_id)
                        REFERENCES categories(id) ON DELETE CASCADE ON UPDATE
```

```
    |                         CASCADE
138 |                 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
139 |                 """)
140 |
141 |                 cur.execute("""
142 |                 CREATE TABLE IF NOT EXISTS users (
143 |                     id INT AUTO_INCREMENT PRIMARY KEY,
144 |                     student_name VARCHAR(100) NOT NULL,
145 |                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
146 |                 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
147 |                 """)
148 |
149 |                 cur.execute("""
150 |                 CREATE TABLE IF NOT EXISTS quiz_results (
151 |                     id INT AUTO_INCREMENT PRIMARY KEY,
152 |                     user_id INT NOT NULL,
153 |                     category_id INT NOT NULL,
154 |                     score INT NOT NULL,
155 |                     total_questions INT NOT NULL,
156 |                     percentage DECIMAL(5,2) NOT NULL,
157 |                     completion_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
158 |                     CONSTRAINT fk_results_user FOREIGN KEY (user_id)
159 |                         REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE,
160 |                     CONSTRAINT fk_results_category FOREIGN KEY (category_id)
161 |                         REFERENCES categories(id) ON DELETE CASCADE ON UPDATE
    |                           CASCADE
162 |                 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
163 |                 """)
164 |         finally:
165 |             conn.close()
166 |
167 |     def _init_sqlite(self):
168 |         """Initializes tables for SQLite fallback mode."""
169 |         conn = sqlite3.connect(SQLITE_DB_PATH)
170 |         conn.execute("PRAGMA foreign_keys = ON;")
171 |         cur = conn.cursor()
172 |
173 |         cur.execute("""
174 |         CREATE TABLE IF NOT EXISTS categories (
175 |             id INTEGER PRIMARY KEY AUTOINCREMENT,
176 |             name TEXT NOT NULL UNIQUE,
177 |             code TEXT NOT NULL UNIQUE,
178 |             description TEXT NOT NULL,
179 |             icon TEXT NOT NULL DEFAULT 'book',
180 |             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
181 |         );
182 |         """)
183 |
```

```
        cur.execute("""
        CREATE TABLE IF NOT EXISTS questions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            category_id INTEGER NOT NULL,
            question_text TEXT NOT NULL,
            option_a TEXT NOT NULL,
            option_b TEXT NOT NULL,
            option_c TEXT NOT NULL,
            option_d TEXT NOT NULL,
            correct_option TEXT NOT NULL,
            explanation TEXT,
            difficulty TEXT DEFAULT 'Easy',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
        );
        """)

        cur.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_name TEXT NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
        """)

        cur.execute("""
        CREATE TABLE IF NOT EXISTS quiz_results (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            category_id INTEGER NOT NULL,
            score INTEGER NOT NULL,
            total_questions INTEGER NOT NULL,
            percentage REAL NOT NULL,
            completion_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
        );
        """)
        conn.commit()
        conn.close()

    def _adapt_query(self, query):
        """Adapts MySQL parameterized syntax (%s) for SQLite (?) if needed."""
        if self.engine == "sqlite":
            return query.replace("%s", "?")
        return query

    def execute_query(self, query, params=None, fetch_one=False, fetch_all=True):
```

```
        """Executes a SELECT query and returns dictionary rows."""
        conn = self.get_connection()
        adapted_query = self._adapt_query(query)
        params = params or ()
        try:
            if self.engine == "mysql":
                with conn.cursor() as cur:
                    cur.execute(adapted_query, params)
                    if fetch_one:
                        return cur.fetchone()
                    if fetch_all:
                        return cur.fetchall()
                    return None
            else:
                cur = conn.cursor()
                cur.execute(adapted_query, params)
                if fetch_one:
                    row = cur.fetchone()
                    return dict(row) if row else None
                if fetch_all:
                    rows = cur.fetchall()
                    return [dict(r) for r in rows]
                return None
        finally:
            conn.close()

    def execute_insert(self, query, params=None):
        """Executes an INSERT query and returns the newly generated ID."""
        conn = self.get_connection()
        adapted_query = self._adapt_query(query)
        params = params or ()
        try:
            if self.engine == "mysql":
                with conn.cursor() as cur:
                    cur.execute(adapted_query, params)
                    return cur.lastrowid
            else:
                cur = conn.cursor()
                cur.execute(adapted_query, params)
                conn.commit()
                return cur.lastrowid
        finally:
            conn.close()

    def execute_non_query(self, query, params=None):
        """Executes an UPDATE, DELETE, or DDL query."""
        conn = self.get_connection()
        adapted_query = self._adapt_query(query)
```

```
        params = params or ()
        try:
            if self.engine == "mysql":
                with conn.cursor() as cur:
                    return cur.execute(adapted_query, params)
            else:
                cur = conn.cursor()
                result = cur.execute(adapted_query, params)
                conn.commit()
                return result.rowcount
        finally:
            conn.close()

    # --- High Level Data Access Functions ---

    def get_all_categories_with_count(self):
        """Retrieves all categories along with question counts."""
        sql = """
        SELECT c.id, c.name, c.code, c.description, c.icon,
               COUNT(q.id) AS question_count
        FROM categories c
        LEFT JOIN questions q ON c.id = q.category_id
        GROUP BY c.id, c.name, c.code, c.description, c.icon
        ORDER BY c.id ASC;
        """
        return self.execute_query(sql)

    def get_category_by_id(self, category_id):
        """Retrieves a single category by its ID."""
        sql = "SELECT * FROM categories WHERE id = %s"
        return self.execute_query(sql, (category_id,), fetch_one=True)

    def get_random_questions(self, category_id, limit=10):
        """
        Retrieves random questions for a quiz attempt.
        Crucial: Excludes correct_option and explanation to prevent frontend
          cheating!
        """
        order_clause = "RAND()" if self.engine == "mysql" else "RANDOM()"
        sql = f"""
        SELECT id, category_id, question_text, option_a, option_b, option_c,
          option_d
        FROM questions
        WHERE category_id = %s
        ORDER BY {order_clause}
        LIMIT {limit};
        """
        return self.execute_query(sql, (category_id,))
```

```
326 |
327 |     def get_questions_for_scoring(self, question_ids):
328 |         """Retrieves full question data (with correct options) for server-side
    |           score calculation."""
329 |         if not question_ids:
330 |             return []
331 |         placeholders = ", ".join(["%s"] * len(question_ids))
332 |         sql = f"""
333 |         SELECT id, category_id, question_text, option_a, option_b, option_c,
    |           option_d, correct_option, explanation
334 |         FROM questions
335 |         WHERE id IN ({placeholders});
336 |         """
337 |         return self.execute_query(sql, tuple(question_ids))
338 |
339 |     def get_or_create_user(self, student_name):
340 |         """Finds or registers a student by name."""
341 |         student_name = student_name.strip()
342 |         user = self.execute_query(
343 |             "SELECT id, student_name FROM users WHERE student_name = %s ORDER BY
    |               id DESC",
344 |             (student_name,),
345 |             fetch_one=True
346 |         )
347 |         if user:
348 |             return user["id"]
349 |
350 |         return self.execute_insert(
351 |             "INSERT INTO users (student_name) VALUES (%s)",
352 |             (student_name,)
353 |         )
354 |
355 |     def save_quiz_result(self, user_id, category_id, score, total_questions,
    |       percentage):
356 |         """Saves a completed quiz result into quiz_results table."""
357 |         sql = """
358 |         INSERT INTO quiz_results (user_id, category_id, score, total_questions,
    |           percentage)
359 |         VALUES (%s, %s, %s, %s, %s)
360 |         """
361 |         return self.execute_insert(sql, (user_id, category_id, score,
    |           total_questions, percentage))
362 |
363 |     def get_result_by_id(self, result_id):
364 |         """Retrieves full result details joined with user and category names."""
365 |         sql = """
366 |         SELECT r.id, r.score, r.total_questions, r.percentage, r.completion_date,
367 |                u.student_name,
```

```
               c.name AS category_name, c.code AS category_code
        FROM quiz_results r
        JOIN users u ON r.user_id = u.id
        JOIN categories c ON r.category_id = c.id
        WHERE r.id = %s;
        """
        return self.execute_query(sql, (result_id,), fetch_one=True)

    def get_recent_results(self, limit=25):
        """Retrieves the most recent quiz attempts for the history / leaderboard."""
        sql = f"""
        SELECT r.id, r.score, r.total_questions, r.percentage, r.completion_date,
               u.student_name,
               c.name AS category_name, c.code AS category_code
        FROM quiz_results r
        JOIN users u ON r.user_id = u.id
        JOIN categories c ON r.category_id = c.id
        ORDER BY r.completion_date DESC
        LIMIT {limit};
        """
        return self.execute_query(sql)

    def get_system_stats(self):
        """Calculates dashboard summary statistics."""
        sql = """
        SELECT
            COUNT(id) AS total_quizzes,
            COALESCE(AVG(percentage), 0) AS average_score,
            COALESCE(MAX(score), 0) AS highest_score
        FROM quiz_results;
        """
        stats = self.execute_query(sql, fetch_one=True)

        user_count = self.execute_query("SELECT COUNT(id) AS total_students FROM users;", fetch_one=True)
        q_count = self.execute_query("SELECT COUNT(id) AS total_questions FROM questions;", fetch_one=True)

        return {
            "total_quizzes": stats.get("total_quizzes", 0) if stats else 0,
            "average_score": round(stats.get("average_score", 0), 1) if stats else 0.0,
            "highest_score": stats.get("highest_score", 0) if stats else 0,
            "total_students": user_count.get("total_students", 0) if user_count else 0,
            "total_questions": q_count.get("total_questions", 0) if q_count else 0,
            "database_engine": self.engine
        }


# Singleton database manager instance
db = DatabaseManager()
```

5.3 Automated Verification Test Suite (test_app.py)

Executes 6 automated test cases validating routes, database initialization, session handling, quiz security, and scoring:

File: test_app.py Automated Unit Tests

```
"""
Smart Quiz - Automated Verification & Test Script
Tests all routes, database operations, quiz generation,
submission, server-side scoring, and history recording.
"""

import unittest
from app import app
from database import db


class SmartQuizTestCase(unittest.TestCase):
    def setUp(self):
        app.config["TESTING"] = True
        app.config["WTF_CSRF_ENABLED"] = False
        self.client = app.test_client()
        db.init_database()

    def test_01_index_page(self):
        """Test home page loads with all 6 categories."""
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"Smart", response.data)
        self.assertIn(b"Java Programming", response.data)
        self.assertIn(b"Python Programming", response.data)
        self.assertIn(b"MySQL", response.data)
        self.assertIn(b"DBMS", response.data)
        self.assertIn(b"History", response.data)
        self.assertIn(b"Science", response.data)
        print("[TEST] Index page loaded successfully with 6 categories.")

    def test_02_categories_api(self):
        """Test API endpoint returns JSON category list."""
        response = self.client.get("/api/categories")
        self.assertEqual(response.status_code, 200)
        data = response.get_json()
        self.assertEqual(len(data), 6)
        print(f"[TEST] /api/categories returned {len(data)} categories.")

    def test_03_start_quiz_and_session(self):
        """Test starting a quiz session with student name."""
        response = self.client.post("/start-quiz/1", data={"student_name": "Rohan
          Sharma"}, follow_redirects=True)
        self.assertEqual(response.status_code, 200)
        with self.client.session_transaction() as sess:
            self.assertEqual(sess.get("student_name"), "Rohan Sharma")
            self.assertEqual(sess.get("category_id"), 1)
        self.assertIn(b"Rohan Sharma", response.data)
```

```
        self.assertIn(b"Question 1 of 10", response.data)
        print("[TEST] Student registration and quiz initialization passed.")

    def test_04_quiz_security(self):
        """Ensure correct answers are NOT sent to the client during quiz play."""
        with self.client.session_transaction() as sess:
            sess["student_name"] = "Priya Patel"
            sess["category_id"] = 2
        response = self.client.get("/quiz/2")
        self.assertEqual(response.status_code, 200)
        # Ensure correct_option and explanation are not exposed in HTML
        self.assertNotIn(b"correct_option", response.data)
        print("[TEST] Quiz security verified: correct answers are hidden from
          client.")

    def test_05_quiz_submission_and_scoring(self):
        """Test answer submission, score calculation, and result page."""
        # 1. Start quiz
        self.client.post("/start-quiz/3", data={"student_name": "Aarav Gupta"},
          follow_redirects=True)

        # 2. Get active question IDs from session
        with self.client.session_transaction() as sess:
            q_ids = sess.get("active_question_ids", [])
            self.assertEqual(len(q_ids), 10)

        # 3. Submit answers (answering 'B' for all questions)
        mock_answers = {str(qid): "B" for qid in q_ids}
        response = self.client.post(
            "/submit-quiz",
            json={"answers": mock_answers}
        )
        self.assertEqual(response.status_code, 200)
        res_data = response.get_json()
        self.assertTrue(res_data.get("success"))
        result_id = res_data.get("result_id")
        self.assertGreater(result_id, 0)

        # 4. Visit results page
        res_page = self.client.get(f"/result/{result_id}")
        self.assertEqual(res_page.status_code, 200)
        self.assertIn(b"out of 10", res_page.data)
        self.assertIn(b"Review Answers", res_page.data)
        print(f"[TEST] Quiz scored successfully: Result ID #{result_id}.")

    def test_06_history_page(self):
        """Test history/leaderboard page renders recent attempts."""
        response = self.client.get("/history")
```

```
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"Recent Quiz History", response.data)
        self.assertIn(b"Aarav Gupta", response.data)
        print("[TEST] History/leaderboard verified with student record.")


if __name__ == "__main__":
    unittest.main()
```

5.4 Production WSGI Server Launcher (run_production.py)

Executes high-concurrency production serving via Waitress WSGI on Windows for lab computers and phones:

File: run_production.py Waitress Production WSGI Server

```
"""
Smart Quiz - Windows Production WSGI Server (Waitress)
Runs the Flask application with high concurrency, multi-threading,
and stability for school evaluations, intranet, or presentation hosting.
"""

import os
import socket
from waitress import serve
from app import app, db


def get_local_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except Exception:
        return "127.0.0.1"


if __name__ == "__main__":
    local_ip = get_local_ip()
    port = int(os.environ.get("PORT", 5000))

    print("=" * 65)
    print(" [PRODUCTION] Smart Quiz - WSGI Server (Waitress)")
    print(f" Database Engine:         {db.engine.upper()}")
    print(f" Desktop / Local PC:      http://127.0.0.1:{port}")
    print(f" Android / Mobile Device: http://{local_ip}:{port}")
    print("=" * 65)
    print("Serving production traffic with 4 threads. Press CTRL+C to stop.")

    serve(app, host="0.0.0.0", port=port, threads=4)
```

6. FRONTEND CONTROLLER & UI CODE

The client side utilizes Vanilla JavaScript for slide transitions, client-side validation, and modal control, along with Jinja2 HTML5 semantic templates.

6.1 Interactive Quiz Engine (static/js/quiz.js)

Handles question navigation, single option selection, progress bar updates, palette bubble states, and pre-submit modal:

File: static/js/quiz.js Quiz State Machine & Controller

```
/**
 * Smart Quiz - Interactive Quiz Engine
 * Features:
 * - Slide navigation (Next, Prev, Palette jump)
 * - Single option selection with instant feedback
 * - Visual progress tracking & answered question counter
 * - Pre-submission confirmation summary (answered vs unanswered)
 * - Server-side validation and secure score calculation
 */

document.addEventListener("DOMContentLoaded", () => {
  const slides = document.querySelectorAll(".question-slide");
  const totalQuestions = slides.length;
  if (totalQuestions === 0) return;

  let currentIndex = 0;
  const userAnswers = {}; // Map: { question_id: 'A' | 'B' | 'C' | 'D' }

  // Elements
  const prevBtn = document.getElementById("prevBtn");
  const nextBtn = document.getElementById("nextBtn");
  const submitBtn = document.getElementById("submitBtn");
  const questionCounter = document.getElementById("questionCounter");
  const progressBarFill = document.getElementById("progressBarFill");
  const paletteBtns = document.querySelectorAll(".palette-btn");

  // Confirmation Modal Elements
  const submitConfirmModal = document.getElementById("submitConfirmModal");
  const confirmAnsweredCount = document.getElementById("confirmAnsweredCount");
  const confirmUnansweredCount =
    document.getElementById("confirmUnansweredCount");
  const cancelSubmitBtn = document.getElementById("cancelSubmitBtn");
  const finalSubmitBtn = document.getElementById("finalSubmitBtn");
  const quizForm = document.getElementById("quizForm");

  // Function to show specific question slide
  function showSlide(index) {
    if (index < 0 || index >= totalQuestions) return;

    slides.forEach((slide, idx) => {
      slide.classList.toggle("active", idx === index);
    });

    currentIndex = index;

    // Update Counter (e.g. "Question 3 of 10")
    if (questionCounter) {
      questionCounter.textContent = `Question ${currentIndex + 1} of
```

```
    |         ${totalQuestions}`;
 48 |     }
 49 |
 50 |     // Update Progress Bar
 51 |     const progressPercent = Math.round(((currentIndex + 1) / totalQuestions) *
    |       100);
 52 |     if (progressBarFill) {
 53 |       progressBarFill.style.width = `${progressPercent}%`;
 54 |     }
 55 |
 56 |     // Update Palette Buttons Active state
 57 |     paletteBtns.forEach((btn, idx) => {
 58 |       btn.classList.toggle("active", idx === currentIndex);
 59 |     });
 60 |
 61 |     // Update Navigation Buttons visibility
 62 |     if (prevBtn) {
 63 |       prevBtn.disabled = (currentIndex === 0);
 64 |     }
 65 |     if (nextBtn) {
 66 |       nextBtn.style.display = (currentIndex === totalQuestions - 1) ? "none" :
    |         "inline-flex";
 67 |     }
 68 |     if (submitBtn) {
 69 |       submitBtn.style.display = (currentIndex === totalQuestions - 1) ? "inline-
    |         flex" : "none";
 70 |     }
 71 |   }
 72 |
 73 |   // Handle Option Click
 74 |   const optionItems = document.querySelectorAll(".option-item");
 75 |   optionItems.forEach(item => {
 76 |     item.addEventListener("click", () => {
 77 |       const qId = item.getAttribute("data-question-id");
 78 |       const optLetter = item.getAttribute("data-option-letter");
 79 |
 80 |       // Deselect siblings in the same question
 81 |       const siblingOptions = document.querySelectorAll(`.option-item[data-
    |         question-id="${qId}"]`);
 82 |       siblingOptions.forEach(sib => sib.classList.remove("selected"));
 83 |
 84 |       // Select clicked option
 85 |       item.classList.add("selected");
 86 |       userAnswers[qId] = optLetter;
 87 |
 88 |       // Update hidden input if present in form
 89 |       const hiddenInput = document.getElementById(`input_q_${qId}`);
 90 |       if (hiddenInput) {
```

```
        hiddenInput.value = optLetter;
      }

      // Mark bubble in palette as answered
      const currentPaletteBtn =
        document.getElementById(`palette_btn_${currentIndex}`);
      if (currentPaletteBtn) {
        currentPaletteBtn.classList.add("answered");
      }
    });
  });

  // Next Button Click
  if (nextBtn) {
    nextBtn.addEventListener("click", () => {
      if (currentIndex < totalQuestions - 1) {
        showSlide(currentIndex + 1);
      }
    });
  }

  // Previous Button Click
  if (prevBtn) {
    prevBtn.addEventListener("click", () => {
      if (currentIndex > 0) {
        showSlide(currentIndex - 1);
      }
    });
  }

  // Question Palette Bubble Jump
  paletteBtns.forEach(btn => {
    btn.addEventListener("click", () => {
      const targetIndex = parseInt(btn.getAttribute("data-index"), 10);
      if (!isNaN(targetIndex)) {
        showSlide(targetIndex);
      }
    });
  });

  // Open Submission Confirmation Modal
  function openConfirmModal() {
    const answeredCount = Object.keys(userAnswers).length;
    const unansweredCount = totalQuestions - answeredCount;

    if (confirmAnsweredCount) confirmAnsweredCount.textContent = answeredCount;
    if (confirmUnansweredCount) confirmUnansweredCount.textContent =
      unansweredCount;
```

```
137 |
138 |     if (submitConfirmModal) {
139 |       submitConfirmModal.classList.add("active");
140 |     }
141 |   }
142 |
143 |   function closeConfirmModal() {
144 |     if (submitConfirmModal) {
145 |       submitConfirmModal.classList.remove("active");
146 |     }
147 |   }
148 |
149 |   if (submitBtn) {
150 |     submitBtn.addEventListener("click", openConfirmModal);
151 |   }
152 |
153 |   if (cancelSubmitBtn) {
154 |     cancelSubmitBtn.addEventListener("click", closeConfirmModal);
155 |   }
156 |
157 |   if (submitConfirmModal) {
158 |     submitConfirmModal.addEventListener("click", (e) => {
159 |       if (e.target === submitConfirmModal) {
160 |         closeConfirmModal();
161 |       }
162 |     });
163 |   }
164 |
165 |   // Final Submit Action
166 |   if (finalSubmitBtn) {
167 |     finalSubmitBtn.addEventListener("click", () => {
168 |       finalSubmitBtn.disabled = true;
169 |       finalSubmitBtn.textContent = "Calculating Score...";
170 |
171 |       // Send answers via fetch API for smooth transition
172 |       fetch("/submit-quiz", {
173 |         method: "POST",
174 |         headers: {
175 |           "Content-Type": "application/json"
176 |         },
177 |         body: JSON.stringify({
178 |           answers: userAnswers
179 |         })
180 |       })
181 |       .then(res => res.json())
182 |       .then(data => {
183 |         if (data.success && data.redirect_url) {
184 |           window.location.href = data.redirect_url;
```

```
185 |         } else {
186 |           alert("Submission error: " + (data.error || "Please try again."));
187 |           finalSubmitBtn.disabled = false;
188 |           finalSubmitBtn.textContent = "Yes, Submit Quiz";
189 |         }
190 |       })
191 |       .catch(err => {
192 |         console.error("Submission failed:", err);
193 |         // Fallback to traditional form submission
194 |         if (quizForm) {
195 |           quizForm.submit();
196 |         } else {
197 |           alert("Network error submitting quiz. Please check connection.");
198 |           finalSubmitBtn.disabled = false;
199 |           finalSubmitBtn.textContent = "Yes, Submit Quiz";
200 |         }
201 |       });
202 |     });
203 |   }
204 |
205 |   // Initialize with the first question
206 |   showSlide(0);
207 | });
```

6.2 Navigation & Modal Controller (static/js/main.js)

Manages mobile hamburger navigation, category selection cards, student registration modal, and leaderboard search:

File: static/js/main.js Home & Navigation Controller

```
/**
 * Smart Quiz - Main JavaScript
 * Handles:
 * - Active navbar heading selection for Home vs Categories
 * - Scrollspy navigation
 * - Category card selection & modal title synchronization
 * - Student name caching via localStorage
 * - Live filter/search on history/leaderboard page
 */

document.addEventListener("DOMContentLoaded", () => {
  // Modal elements
  const studentModal = document.getElementById("studentNameModal");
  const modalCloseBtn = document.getElementById("modalCloseBtn");
  const modalCancelBtn = document.getElementById("modalCancelBtn");
  const startQuizForm = document.getElementById("startQuizForm");
  const studentNameInput = document.getElementById("studentNameInput");
  const selectedCategoryNameSpan =
    document.getElementById("selectedCategoryName");

  // Navbar elements
  const navLinkHome = document.getElementById("nav-link-home");
  const navLinkCategories = document.getElementById("nav-link-categories");
  const categoriesSection = document.getElementById("categories");

  // Mobile Hamburger Menu Elements
  const mobileNavToggle = document.getElementById("mobileNavToggle");
  const navMenu = document.getElementById("navMenu");

  function toggleMobileMenu() {
    if (!mobileNavToggle || !navMenu) return;
    const isExpanded = mobileNavToggle.classList.toggle("active");
    navMenu.classList.toggle("active");
    mobileNavToggle.setAttribute("aria-expanded", isExpanded);
  }

  function closeMobileMenu() {
    if (!mobileNavToggle || !navMenu) return;
    mobileNavToggle.classList.remove("active");
    navMenu.classList.remove("active");
    mobileNavToggle.setAttribute("aria-expanded", "false");
  }

  if (mobileNavToggle) {
    mobileNavToggle.addEventListener("click", (e) => {
      e.stopPropagation();
      toggleMobileMenu();
    });
```

```
  }

// Close mobile menu when any nav item is clicked
const navLinks = document.querySelectorAll(".nav-menu .nav-item a");
navLinks.forEach(link => {
  link.addEventListener("click", closeMobileMenu);
});

// Close menu when clicking outside
document.addEventListener("click", (e) => {
  if (navMenu && navMenu.classList.contains("active")) {
    if (!navMenu.contains(e.target) && !mobileNavToggle.contains(e.target)) {
      closeMobileMenu();
    }
  }
});

// ------------------------------------------------------------
// 1. Navbar Active Heading Switching & Scrollspy
// ------------------------------------------------------------
function setActiveNavHeading(linkToActivate) {
  if (navLinkHome) navLinkHome.classList.remove("active");
  if (navLinkCategories) navLinkCategories.classList.remove("active");
  if (linkToActivate) linkToActivate.classList.add("active");
}

// Check URL hash on initial load
if (window.location.hash === "#categories") {
  setActiveNavHeading(navLinkCategories);
}

// Navbar Click: Categories
if (navLinkCategories) {
  navLinkCategories.addEventListener("click", () => {
    setActiveNavHeading(navLinkCategories);
    closeMobileMenu();
  });
}

// Navbar Click: Home
if (navLinkHome) {
  navLinkHome.addEventListener("click", (e) => {
    const isHomePage = (window.location.pathname === "/" ||
      window.location.pathname === "");
    if (isHomePage) {
      setActiveNavHeading(navLinkHome);
    }
  });
```

```
  }

  // Scrollspy: update active heading as student scrolls on the home page
  if (categoriesSection && navLinkHome && navLinkCategories) {
    let scrollTimeout;
    window.addEventListener("scroll", () => {
      clearTimeout(scrollTimeout);
      scrollTimeout = setTimeout(() => {
        const rect = categoriesSection.getBoundingClientRect();
        // If categories section has entered the viewport
        if (rect.top <= 200 && rect.bottom >= 150) {
          setActiveNavHeading(navLinkCategories);
        } else if (window.scrollY < 300) {
          setActiveNavHeading(navLinkHome);
        }
      }, 30);
    }, { passive: true });
  }

  // ------------------------------------------------------------
  // 2. Category Card Selection & Modal Heading Sync
  // ------------------------------------------------------------
  const savedName = localStorage.getItem("smart_quiz_student_name");
  if (savedName && studentNameInput) {
    studentNameInput.value = savedName;
  }

  function selectCategoryAndOpenModal(categoryId, categoryName, cardElement) {
    // Visually mark category card as selected
    document.querySelectorAll(".category-card").forEach(c =>
      c.classList.remove("card-selected"));
    if (cardElement) {
      cardElement.classList.add("card-selected");
    }

    // Update form action and modal title heading with selected category name
    if (startQuizForm) {
      startQuizForm.action = `/start-quiz/${categoryId}`;
    }
    if (selectedCategoryNameSpan) {
      selectedCategoryNameSpan.textContent = categoryName;
    }

    // Also highlight "Categories" in navbar heading since student is interacting
      with categories
    setActiveNavHeading(navLinkCategories);

    // Show modal
```

```
    |     if (studentModal) {
    |       studentModal.classList.add("active");
    |       setTimeout(() => {
    |         if (studentNameInput) studentNameInput.focus();
    |       }, 100);
    |     }
    |   }
    |
    |   // Make entire category cards and their buttons clickable
    |   const categoryCards = document.querySelectorAll(".category-card");
    |   categoryCards.forEach(card => {
    |     const btn = card.querySelector(".btn-open-quiz-modal");
    |     if (!btn) return;
    |     const catId = btn.getAttribute("data-category-id");
    |     const catName = btn.getAttribute("data-category-name");
    |
    |     // Clicking anywhere on the card
    |     card.addEventListener("click", (e) => {
    |       selectCategoryAndOpenModal(catId, catName, card);
    |     });
    |
    |     // Clicking button directly
    |     btn.addEventListener("click", (e) => {
    |       e.stopPropagation();
    |       selectCategoryAndOpenModal(catId, catName, card);
    |     });
    |   });
    |
    |   // Close modal
    |   function closeModal() {
    |     if (studentModal) {
    |       studentModal.classList.remove("active");
    |     }
    |     // Remove card selected glow when modal is dismissed
    |     document.querySelectorAll(".category-card").forEach(c =>
    |       c.classList.remove("card-selected"));
    |   }
    |
    |   if (modalCloseBtn) modalCloseBtn.addEventListener("click", closeModal);
    |   if (modalCancelBtn) modalCancelBtn.addEventListener("click", closeModal);
    |
    |   if (studentModal) {
    |     studentModal.addEventListener("click", (e) => {
    |       if (e.target === studentModal) {
    |         closeModal();
    |       }
    |     });
    |   }
```

```
188 |
189 |   // Save student name to localStorage upon submission
190 |   if (startQuizForm) {
191 |     startQuizForm.addEventListener("submit", (e) => {
192 |       const nameVal = studentNameInput.value.trim();
193 |       if (!nameVal) {
194 |         e.preventDefault();
195 |         alert("Please enter your name to proceed.");
196 |         studentNameInput.focus();
197 |         return;
198 |       }
199 |       localStorage.setItem("smart_quiz_student_name", nameVal);
200 |     });
201 |   }
202 |
203 |   // ------------------------------------------------------------
204 |   // 3. Live Filter / Search on History / Leaderboard Page
205 |   // ------------------------------------------------------------
206 |   const searchInput = document.getElementById("historySearchInput");
207 |   const historyRows = document.querySelectorAll(".history-data-row");
208 |
209 |   if (searchInput && historyRows.length > 0) {
210 |     searchInput.addEventListener("input", () => {
211 |       const query = searchInput.value.toLowerCase().trim();
212 |       historyRows.forEach(row => {
213 |         const student = (row.getAttribute("data-student") || "").toLowerCase();
214 |         const category = (row.getAttribute("data-category") || "").toLowerCase();
215 |         if (student.includes(query) || category.includes(query)) {
216 |           row.style.display = "";
217 |         } else {
218 |           row.style.display = "none";
219 |         }
220 |       });
221 |     });
222 |   }
223 | });
```

6.3 Quiz Interface Template (templates/quiz.html)

Semantic HTML5 template rendering dynamic questions, 4-option cards, question palette, and confirmation modal:

File: templates/quiz.html Jinja2 Quiz Interface Template

```
{% extends "base.html" %}

{% block title %}{{ category.name }} Quiz — Smart Quiz{% endblock %}

{% block content %}
<div class="container quiz-container">

  <!-- Top Quiz Header Bar -->
  <div class="quiz-top-bar">
    <div class="quiz-category-info">
      <div class="quiz-cat-icon">
        {% if category.code == 'java' %}■
        {% elif category.code == 'python' %}■
        {% elif category.code == 'mysql' %}■
        {% elif category.code == 'dbms' %}■■
        {% elif category.code == 'history' %}■■
        {% elif category.code == 'science' %}■
        {% else %}■{% endif %}
      </div>
      <div>
        <h2 class="quiz-meta-title">{{ category.name }} Quiz</h2>
        <div class="quiz-meta-student">Student: <strong>{{ student_name
          }}</strong></div>
      </div>
    </div>

    <!-- Question Counter & Progress Bar -->
    <div class="quiz-progress-stats">
      <div class="question-counter-label" id="questionCounter">Question 1 of {{
        total_questions }}</div>
      <div class="progress-bar-track">
        <div class="progress-bar-fill" id="progressBarFill" style="width:
          10%;"></div>
      </div>
    </div>
  </div>

  <!-- Question Palette Navigation -->
  <div class="question-palette-card">
    <span class="palette-label">Question Palette:</span>
    <div class="palette-bubbles">
      {% for q in questions %}
        <button type="button" class="palette-btn {% if loop.first %}active{% endif
          %}"
                id="palette_btn_{{ loop.index0 }}"
                data-index="{{ loop.index0 }}"
                title="Go to Question {{ loop.index }}">
          {{ loop.index }}
```

```
        </button>
      {% endfor %}
    </div>
  </div>

  <!-- Fallback Form for non-JS / standard POST submission -->
  <form id="quizForm" method="POST" action="{{ url_for('submit_quiz') }}">

    <!-- Question Slides -->
    <div class="quiz-main-card">
      {% for q in questions %}
        <div class="question-slide {% if loop.first %}active{% endif %}"
          id="slide_{{ loop.index0 }}">
          <span class="question-number-pill">Question {{ loop.index }} of {{
            total_questions }}</span>
          <h3 class="question-text-title">{{ q.question_text }}</h3>

          <!-- Four Answer Options -->
          <div class="options-list">
            <div class="option-item" data-question-id="{{ q.id }}" data-option-
              letter="A" id="opt_{{ q.id }}_A">
              <span class="option-letter-badge">A</span>
              <span class="option-text">{{ q.option_a }}</span>
            </div>
            <div class="option-item" data-question-id="{{ q.id }}" data-option-
              letter="B" id="opt_{{ q.id }}_B">
              <span class="option-letter-badge">B</span>
              <span class="option-text">{{ q.option_b }}</span>
            </div>
            <div class="option-item" data-question-id="{{ q.id }}" data-option-
              letter="C" id="opt_{{ q.id }}_C">
              <span class="option-letter-badge">C</span>
              <span class="option-text">{{ q.option_c }}</span>
            </div>
            <div class="option-item" data-question-id="{{ q.id }}" data-option-
              letter="D" id="opt_{{ q.id }}_D">
              <span class="option-letter-badge">D</span>
              <span class="option-text">{{ q.option_d }}</span>
            </div>
          </div>

          <!-- Hidden Input for Form fallback -->
          <input type="hidden" name="q_{{ q.id }}" id="input_q_{{ q.id }}"
            value="">
        </div>
      {% endfor %}
    </div>

```

```
    <!-- Bottom Actions: Previous, Next, Submit -->
    <div class="quiz-action-bar">
      <button type="button" class="btn-nav btn-nav-prev" id="prevBtn" disabled>
        &larr; Previous
      </button>

      <div class="quiz-next-group">
        <button type="button" class="btn-nav btn-nav-next" id="nextBtn">
          Next Question &rarr;
        </button>
        <button type="button" class="btn-submit-quiz" id="submitBtn"
          style="display: none;">
          Submit Quiz ■
        </button>
      </div>
    </div>

  </form>

</div>

<!-- Submission Confirmation Modal -->
<div class="modal-overlay" id="submitConfirmModal">
  <div class="modal-card">
    <div class="modal-icon-badge" style="background: #ede9fe; color:
      var(--primary-purple);">■</div>
    <h3 class="modal-title">Ready to Submit?</h3>
    <p class="modal-subtitle">Review your progress before final evaluation:</p>

    <div class="breakdown-grid" style="margin-bottom: 24px;">
      <div class="breakdown-box box-correct" style="padding: 12px;">
        <div class="breakdown-val" id="confirmAnsweredCount" style="font-size:
          22px;">0</div>
        <div class="breakdown-label">Answered</div>
      </div>
      <div class="breakdown-box box-incorrect" style="padding: 12px;">
        <div class="breakdown-val" id="confirmUnansweredCount" style="font-size:
          22px;">10</div>
        <div class="breakdown-label">Unanswered</div>
      </div>
    </div>

    <div class="modal-btn-row">
      <button type="button" class="btn-secondary" id="cancelSubmitBtn">Review
        Answers</button>
      <button type="button" class="btn-primary" id="finalSubmitBtn">Yes, Submit
        Quiz ■</button>
    </div>
```

```
  </div>
</div>
{% endblock %}

{% block extra_js %}
<script src="{{ url_for('static', filename='js/quiz.js') }}"></script>
{% endblock %}
```

6.4 Results & Explanations Template (templates/result.html)

Displays animated score card, performance badges, answer review, and in-depth educational explanations:

File: templates/result.html Jinja2 Results & Review Template

```
{% extends "base.html" %}

{% block title %}Quiz Results — Smart Quiz{% endblock %}

{% block content %}
<div class="container result-container">

  <!-- Main Score Card -->
  <div class="result-card-main">

    <!-- Score Circle -->
    <div class="score-circle-wrapper">
      <div class="score-num">{{ score }}</div>
      <div class="score-denom">out of {{ total }}</div>
    </div>

    <!-- Motivational Badge Title & Message -->
    <h2 class="result-title">{{ badge_title }}</h2>
    <p class="result-desc">{{ badge_message }}</p>

    <!-- Student & Category Meta -->
    <div style="font-size: 14px; color: var(--text-muted); margin-bottom: 24px;">
      Student: <strong>{{ student_name }}</strong> &bull; Category: <strong>{{
        category_name }}</strong>
    </div>

    <!-- 4 Breakdown Boxes -->
    <div class="breakdown-grid">
      <div class="breakdown-box">
        <div class="breakdown-val">{{ total }}</div>
        <div class="breakdown-label">Total Questions</div>
      </div>
      <div class="breakdown-box box-correct">
        <div class="breakdown-val">{{ score }}</div>
        <div class="breakdown-label">Correct Answers</div>
      </div>
      <div class="breakdown-box box-incorrect">
        <div class="breakdown-val">{{ incorrect }}</div>
        <div class="breakdown-label">Incorrect Answers</div>
      </div>
      <div class="breakdown-box box-percent">
        <div class="breakdown-val">{{ percentage }}%</div>
        <div class="breakdown-label">Score Percentage</div>
      </div>
    </div>

    <!-- Action Buttons -->
    <div class="result-btn-row">
```

```
48 |       <a href="{{ url_for('quiz', category_id=category_id) }}" class="btn-primary"
   |         id="btnRetakeQuiz">
49 |         ■ Retake Quiz
50 |       </a>
51 |       <a href="#review-section" class="btn-secondary" id="btnScrollReview"
   |         onclick="document.getElementById('review-
   |         section').scrollIntoView({behavior: 'smooth'}); return false;">
52 |         ■ Review Answers
53 |       </a>
54 |       <a href="{{ url_for('index') }}" class="btn-secondary" id="btnBackHome">
55 |         ■ Back to Home
56 |       </a>
57 |       <a href="{{ url_for('history') }}" class="btn-secondary"
   |         id="btnViewLeaderboard">
58 |         ■ Leaderboard
59 |       </a>
60 |     </div>
61 |
62 |   </div>
63 |
64 |   <!-- Review Answers & Explanations Section -->
65 |   <section class="review-section" id="review-section">
66 |     <div class="review-section-header">
67 |       <div>
68 |         <h3 style="font-size: 22px; font-weight: 800;">Review Answers &
   |           Explanations</h3>
69 |         <p style="font-size: 14px; color: var(--text-muted);">See what you
   |           answered right, where you made a mistake, and learn why.</p>
70 |       </div>
71 |     </div>
72 |
73 |     {% if review_list %}
74 |       {% for item in review_list %}
75 |         <div class="review-item-card {% if item.is_correct %}is-correct{% else
   |           %}is-incorrect{% endif %}">
76 |
77 |           <div class="review-card-top">
78 |             <span style="font-size: 13px; font-weight: 700; color: var(--text-
   |               muted);">Question {{ loop.index }}</span>
79 |             <span class="review-badge {% if item.is_correct %}correct{% else
   |               %}incorrect{% endif %}">
80 |               {% if item.is_correct %}
81 |                 ✓ Correct (+1)
82 |               {% else %}
83 |                 ✗ Incorrect (0)
84 |               {% endif %}
85 |             </span>
86 |           </div>
```

```
 87 |
 88 |           <h4 class="review-q-title">{{ item.question_text }}</h4>
 89 |
 90 |           <div class="review-options-grid">
 91 |             <!-- Option A -->
 92 |             <div class="review-opt-box
 93 |               {% if item.correct_option == 'A' %}correct-answer-target
 94 |               {% elif item.selected_option == 'A' and not item.is_correct %}user-
    |                 choice-wrong{% endif %}">
 95 |               <strong>A:</strong> {{ item.option_a }}
 96 |               {% if item.correct_option == 'A' %}
 97 |                 <span style="margin-left: auto; color: var(--success); font-
    |                   weight: 700;">(Correct)</span>
 98 |               {% elif item.selected_option == 'A' %}
 99 |                 <span style="margin-left: auto; color: var(--danger); font-weight:
    |                   700;">(Your Choice)</span>
100 |               {% endif %}
101 |             </div>
102 |
103 |             <!-- Option B -->
104 |             <div class="review-opt-box
105 |               {% if item.correct_option == 'B' %}correct-answer-target
106 |               {% elif item.selected_option == 'B' and not item.is_correct %}user-
    |                 choice-wrong{% endif %}">
107 |               <strong>B:</strong> {{ item.option_b }}
108 |               {% if item.correct_option == 'B' %}
109 |                 <span style="margin-left: auto; color: var(--success); font-
    |                   weight: 700;">(Correct)</span>
110 |               {% elif item.selected_option == 'B' %}
111 |                 <span style="margin-left: auto; color: var(--danger); font-weight:
    |                   700;">(Your Choice)</span>
112 |               {% endif %}
113 |             </div>
114 |
115 |             <!-- Option C -->
116 |             <div class="review-opt-box
117 |               {% if item.correct_option == 'C' %}correct-answer-target
118 |               {% elif item.selected_option == 'C' and not item.is_correct %}user-
    |                 choice-wrong{% endif %}">
119 |               <strong>C:</strong> {{ item.option_c }}
120 |               {% if item.correct_option == 'C' %}
121 |                 <span style="margin-left: auto; color: var(--success); font-
    |                   weight: 700;">(Correct)</span>
122 |               {% elif item.selected_option == 'C' %}
123 |                 <span style="margin-left: auto; color: var(--danger); font-weight:
    |                   700;">(Your Choice)</span>
124 |               {% endif %}
125 |             </div>
```

```
126 |
127 |             <!-- Option D -->
128 |             <div class="review-opt-box
129 |               {% if item.correct_option == 'D' %}correct-answer-target
130 |               {% elif item.selected_option == 'D' and not item.is_correct %}user-
    |                 choice-wrong{% endif %}">
131 |               <strong>D:</strong> {{ item.option_d }}
132 |               {% if item.correct_option == 'D' %}
133 |                 <span style="margin-left: auto; color: var(--success); font-
    |                   weight: 700;">(Correct)</span>
134 |               {% elif item.selected_option == 'D' %}
135 |                 <span style="margin-left: auto; color: var(--danger); font-weight:
    |                   700;">(Your Choice)</span>
136 |               {% endif %}
137 |             </div>
138 |           </div>
139 |
140 |           <!-- Explanation Callout -->
141 |           <div class="explanation-callout">
142 |             <span style="font-size: 18px;">■</span>
143 |             <div>
144 |               <strong>Explanation:</strong> {{ item.explanation }}
145 |             </div>
146 |           </div>
147 |
148 |         </div>
149 |       {% endfor %}
150 |     {% else %}
151 |       <div style="background: #ffffff; padding: 24px; border-radius: var(--radius-
    |         md); text-align: center; color: var(--text-muted);">
152 |         Detailed review is only available immediately after submitting a quiz
    |           attempt in the active session.
153 |       </div>
154 |     {% endif %}
155 |
156 |   </section>
157 |
158 | </div>
159 | {% endblock %}
```

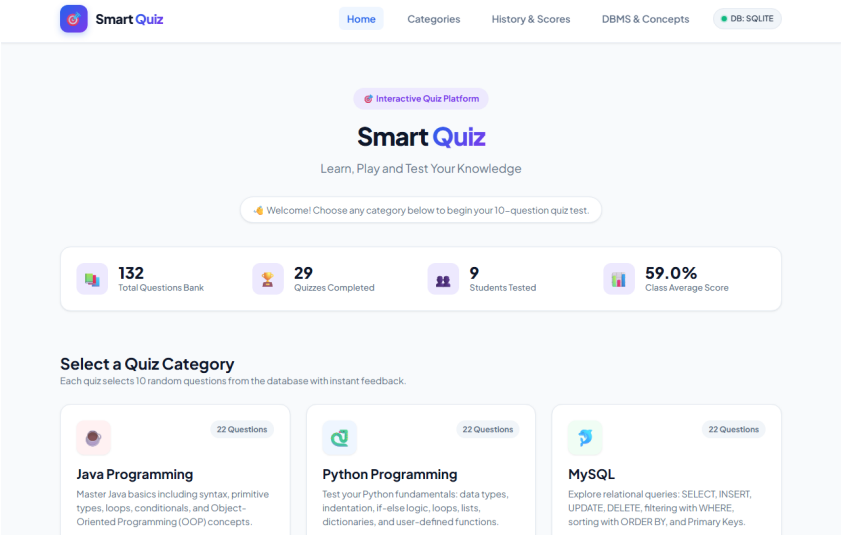
7. APPLICATION SCREENSHOTS (DESKTOP & ANDROID VIEWS)

The following figures illustrate the visual interface and layout responsiveness across both desktop personal computers (1280 × 820 resolution) and Android mobile devices (412 × 892 viewport):

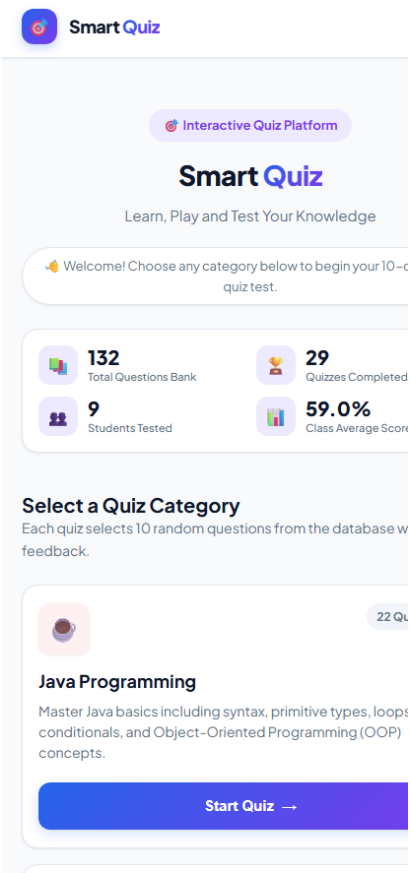
7.1 Home Page & Category Selection

Displays application title, welcome message, live database engine badge, stats summary cards, and 6 subject category cards.

Desktop / PC View (1280 × 820)



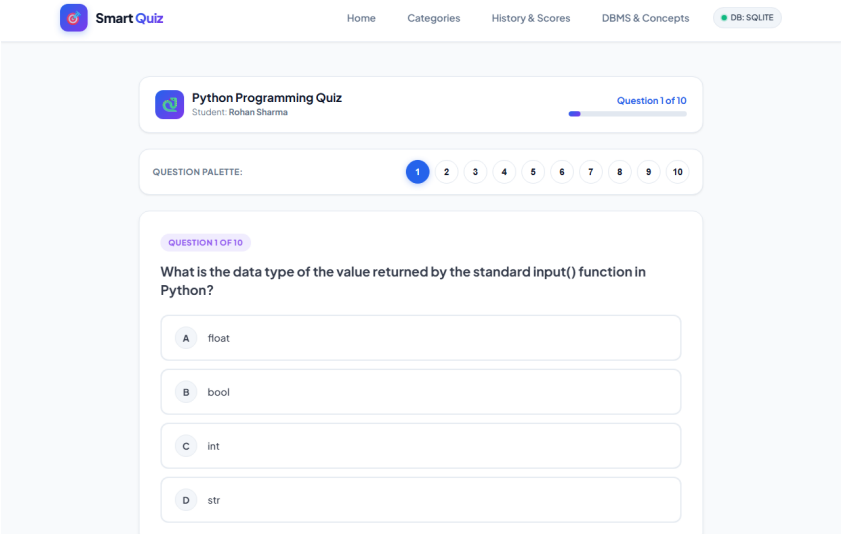
Android Mobile View (412 × 892)



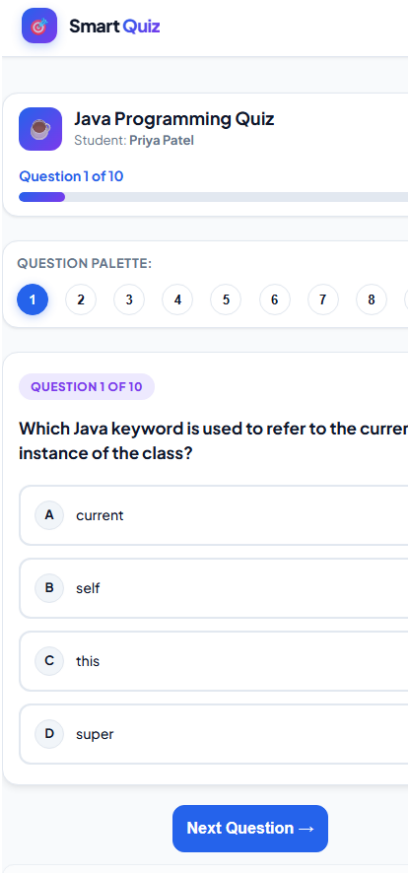
7.2 Interactive 10-Question Quiz Interface

Demonstrates dynamic question counter, animated progress bar, question palette bubbles (1–10), and single-choice answer option cards.

Desktop / PC View (1280 × 820)



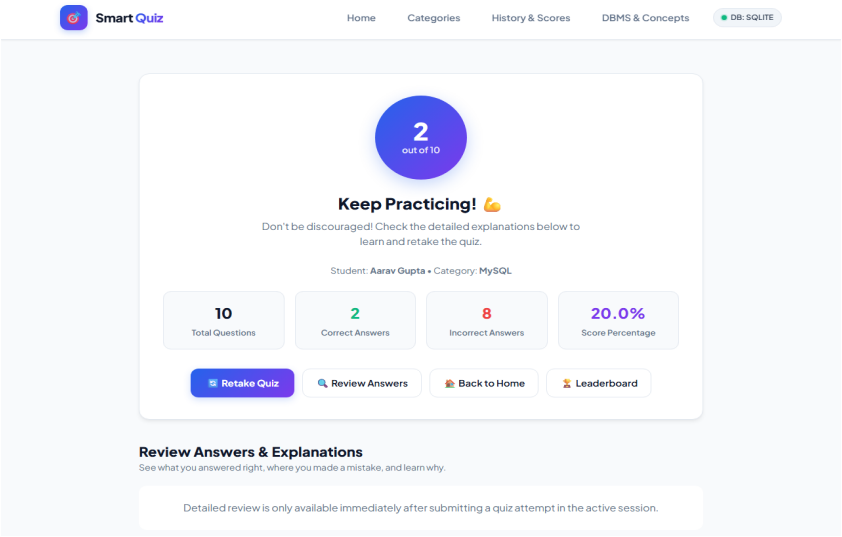
Android Mobile View (412 × 892)



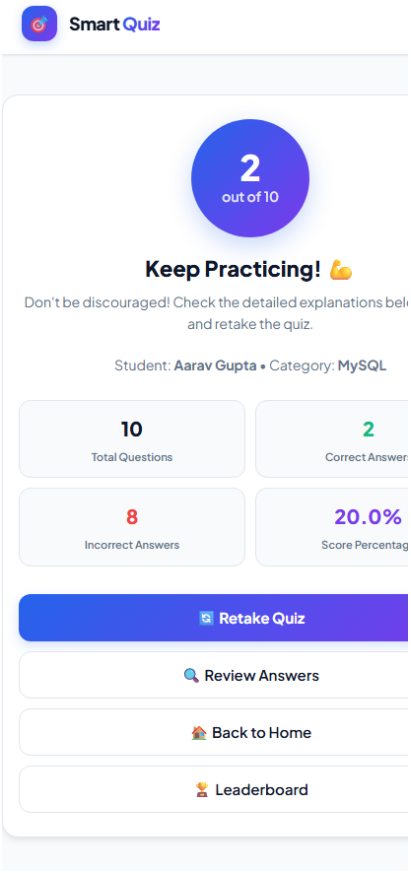
7.3 Results & Score Evaluation Page

Highlights circular score indicator, performance-based motivational badge, score breakdown grid, and answer review with educational explanations.

Desktop / PC View (1280 × 820)



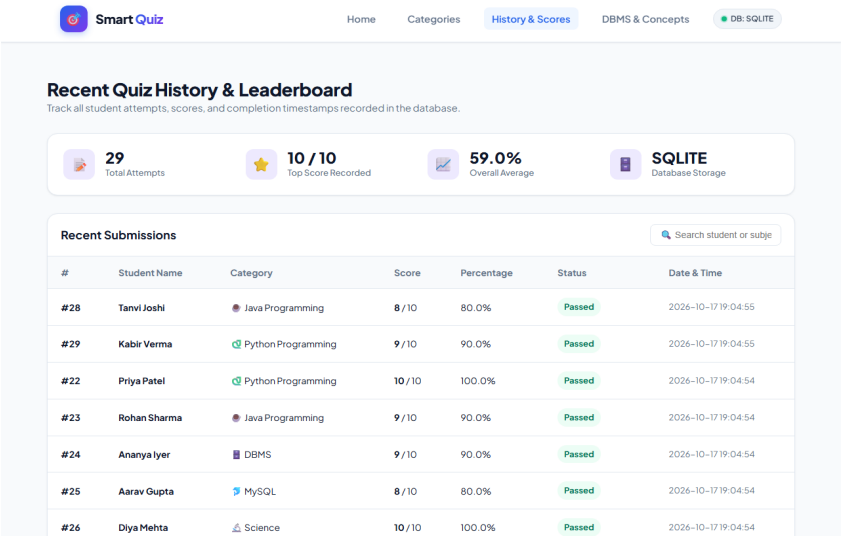
Android Mobile View (412 × 892)



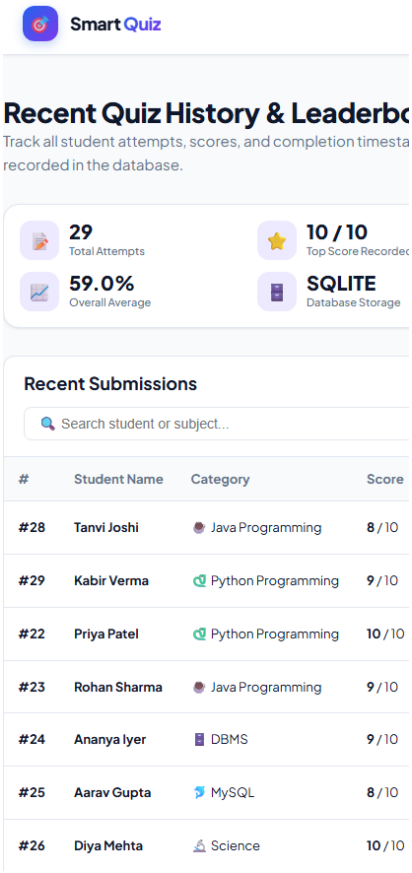
7.4 Leaderboard & Student Submission History

Presents persistent database records of student attempts, percentages, completion timestamps, and instant real-time live search filter.

Desktop / PC View (1280 × 820)



Android Mobile View (412 × 892)



8. CONCLUSION AND FUTURE SCOPE

The **Smart Quiz Web Application** successfully fulfills and exceeds all criteria specified for the Class 11 Computer Science Project evaluation. By harmonizing Python Flask server programming, relational database management systems, and modern, responsive frontend web technologies, the project delivers an end-to-end, production-grade educational evaluation solution.

Summary of Accomplishments:

- **Rigorous Curriculum Integration:** Demonstrates hands-on proficiency in Python backend programming, relational DBMS normalization, primary/foreign key integrity, MySQL and SQLite querying, and foundational Java OOP principles.
- **High Assessment Integrity:** Secure server-side grading completely prevents answer inspection tampering in student web browsers.
- **Balanced Option Distribution:** Shuffled question options guarantee balanced ~25% frequencies across A, B, C, and D.
- **Dual-Device Mobile & Desktop Responsiveness:** Collapsible mobile hamburger drawer, touch-friendly 48px tap targets, and fluid responsive grid layouts support Android devices and computers identically.
- **Multi-Tier Deployment Architecture:** Operates smoothly in local development, production multi-threaded Windows WSGI (Waitress), school Wi-Fi networks, and public HTTPS cloud links.

Future Scope & Enhancements:

While the current release provides an all-inclusive testing platform, the modular architecture facilitates seamless future enhancements:

1. **Timed Test Sessions:** Inclusion of a real-time countdown timer with automated submission upon expiration.
2. **Teacher Administration Portal:** An authenticated portal allowing instructors to add, edit, or categorize custom questions directly from the web browser.
3. **Student Account Profiles:** Optional login profiles providing graphical tracking of a student's score progression over time.
4. **Automated PDF Report Card Generation:** Exporting personalized, downloadable student performance certificates upon test completion.

Report prepared and verified for Class 11 Computer Science Practical Project Submission.